

Traffic Flow Detection Using Artificial, Convolutional and Quantum Neural Networks

Riyan Wankhede (23BCE9287)
Department of Computer Science and Engineering
VIT-AP University
Vellore Institute of Technology, Andhra Pradesh
riyanwankhede@gmail.com

Abstract - Short-term prediction of traffic conditions is a core task in intelligent transportation systems. This paper compares an artificial neural network (ANN), a one-dimensional convolutional neural network (1-D CNN) and a hybrid quantum neural network (QNN) for predicting the congestion level of individual freeway detectors in the real-world PeMS08 dataset. Each sample is a two-hour window of flow, occupancy and speed from one of 170 loop detectors, and the target is that detector's flow class (Low, Medium or High, defined by its own training-period percentiles) five minutes ahead. All models are evaluated on a chronologically held-out test period of 606,390 windows and compared with simple baselines. Persistence-type rules prove strong: a 15-minute moving-average rule reaches 88.08% accuracy. The 1-D CNN performs best (89.18% accuracy, macro-F1 0.891), followed by the ANN (88.99%); both exceed the best rule by about one percentage point and do so on 121 and 108 of the 170 detectors, respectively. The six-qubit QNN, limited by simulation cost to six principal components and 50,000 training windows, reaches 86.25%, on par with a size-matched classical control (86.13%). The results show that learned models add a small but consistent gain over persistence for five-minute-ahead congestion prediction, and that the simulated quantum layer offers no measurable advantage over an equivalent classical layer in this setting.

Keywords - traffic flow prediction, congestion classification, PeMS08, artificial neural network, convolutional neural network, quantum neural network, variational quantum circuit, persistence baseline

I. INTRODUCTION

Growing vehicle numbers have made congestion one of the costliest problems of urban and freeway networks. Accurate short-term predictions of traffic conditions allow operators to adjust ramp metering and signal timing, warn drivers and plan incident response before congestion forms. Freeway networks in California are instrumented with thousands of inductive loop detectors that report vehicle flow, occupancy and speed every few minutes [1], and these measurements have become the standard testbed for data-driven traffic forecasting [2].

Forecasting methods have evolved from statistical time-series models [3] and kernel methods [4] to deep neural networks [5], [6], convolutional models that treat traffic as images [7], [8] and graph neural networks that exploit the road topology [9], [10], [11]. In parallel, variational quantum circuits have been proposed as trainable layers inside hybrid quantum-classical networks [12], [13], [14], raising the question of whether such quantum neural networks (QNNs) are useful for practical prediction tasks such as traffic.

A recurring difficulty in comparing such models is evaluation. Consecutive traffic readings are highly correlated, so a naive rule that assumes conditions stay the same is already accurate at short horizons, and randomly splitting overlapping time windows into training and test sets lets a model score well by memorisation [2]. Reported accuracies are therefore only meaningful relative to strong naive baselines and under a strictly chronological evaluation.

This paper studies five-minute-ahead congestion classification for individual detectors of the PeMS08 dataset [11] under such a protocol. The main contributions are:

- 1) A leakage-free, per-detector formulation of congestion classification with a chronological train/validation/test split, in which class thresholds and feature scaling are computed from the training period only.
- 2) A comparison of an ANN, a 1-D CNN and a hybrid PennyLane QNN against five baselines, including persistence and moving-average rules, on 606,390 test windows.
- 3) A size-matched classical control for the QNN that isolates the contribution of the quantum layer from that of the surrounding network and input compression.
- 4) A per-detector analysis and paired significance tests that show where the learned models help, where they fail, and why.

II. RELATED WORK

A. Statistical and Classical Machine Learning Methods

Early short-term forecasting relied on statistical time-series models. Williams and Hoel [3] showed that freeway flow can be modelled as a seasonal ARIMA process, and Wu et al. [4] applied support vector regression to travel-time prediction. Vlahogianni et al. [2] review this literature and stress the importance of realistic evaluation, including comparison with naive predictors.

B. Deep Learning for Traffic Forecasting

Lv et al. [5] used stacked autoencoders to predict traffic flow from large-scale detector data, and Polson and Sokolov [6] showed that deep networks capture sharp regime changes such as the onset of congestion. Convolutional networks were applied by treating network-wide speeds as images [7] and by

modelling citywide crowd flows with residual convolutional units [8]. Graph-based models such as DCRNN [9], STGCN [10] and ASTGCN [11] exploit the detector network topology; ASTGCN also introduced the PeMS04 and PeMS08 benchmarks used here. These works mostly address multi-step regression of flow values; this paper instead studies per-detector classification of congestion levels, which allows a direct comparison with simple rules.

C. Quantum Neural Networks

Quantum neural networks aim to combine the representational properties of quantum systems with neural learning [12], [13]. In the variational or “circuit learning” approach [14], classical inputs are encoded as rotation angles, a parameterised circuit transforms the state, and measured expectation values feed a classical model; circuit-centric classifiers built from strongly entangling layers [15] are a common design, and PennyLane [16] enables training such circuits with standard automatic differentiation. Current devices and simulators restrict circuits to few qubits [17], and training can suffer from vanishing gradients [18], so fair comparisons against classical models of similar size are essential. Applications of QNNs to traffic data remain comparatively rare.

III. DATASET

PeMS08 contains data from 170 loop detectors on freeways in the San Bernardino area of California (Caltrans District 8), collected by the Caltrans Performance Measurement System [1] and released as a benchmark with ASTGCN [11]. It covers 1 July to 31 August 2016 (62 days) at a five-minute resolution, giving a tensor of $17,856 \text{ timesteps} \times 170 \text{ detectors} \times 3 \text{ features}$:

- **Flow:** number of vehicles passing the detector in the five-minute interval;
- **Occupancy:** fraction of the interval (0–1) during which a vehicle was over the detector;
- **Speed:** average speed of passing vehicles in miles per hour.

The released tensor contains no missing values. Two properties shape the task. First, detectors differ strongly in traffic volume: average flow ranges from about 9 to 461 vehicles per five minutes, a factor of about 50 between the quietest and busiest detectors. Second, traffic is highly persistent at short horizons: the correlation between a detector’s flow in consecutive five-minute intervals is 0.984. The data also show clear daily and weekly cycles, with morning and afternoon peaks on weekdays and a single broad midday peak at weekends.

IV. METHODOLOGY

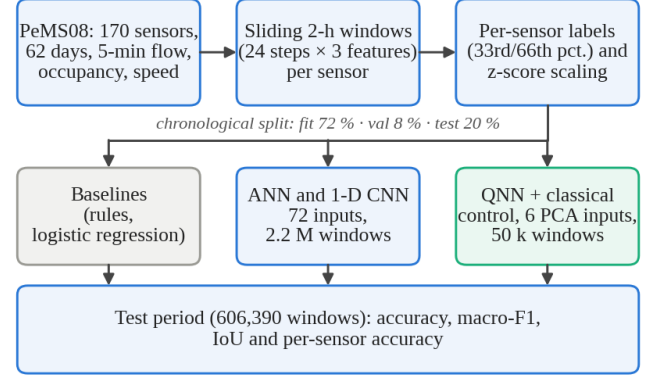


Fig. 1. Overview of the pipeline. Windows, labels and scaling are computed per detector; thresholds and scaling statistics use the training period only.

A. Problem Formulation

For detector s and time t , the input is the window of the previous 24 readings (two hours) of flow, occupancy and speed, $X_{s,t} \in \mathbb{R}^{24 \times 3}$. The target is the congestion class of the detector’s flow $f_{s,t+1}$ in the next five-minute interval:

$$\begin{aligned}
 y_{s,t} &= 0 \text{ (Low) if } f_{s,t+1} \leq q_s^{33}, \\
 y_{s,t} &= 1 \text{ (Medium) if } q_s^{33} < f_{s,t+1} \leq q_s^{66}, \\
 y_{s,t} &= 2 \text{ (High) if } f_{s,t+1} > q_s^{66},
 \end{aligned}$$

where q_s^{33} and q_s^{66} are the 33rd and 66th percentiles of detector s ’s flow during the training period. Detector-specific thresholds make “High” mean busy *for that location*, so a quiet ramp and a busy freeway are judged on their own scales, and they yield three approximately balanced classes.

B. Chronological Split and Preprocessing

Sliding windows with a stride of one step give 17,832 windows per detector. The first 80% of the timeline (14,265 windows per detector, 2,425,050 samples in total) forms the training set and the last 20% (3,567 per detector, 606,390 samples) the test set. The training period covers timesteps 0–14,288, about 49.6 days; class shares are 33.1/33.2/33.7% (Low/Medium/High) in training and 33.7/32.7/33.6% in testing. The latest 10% of the training timeline (242,590 samples) is held out for validation and early stopping, leaving 2,182,460 samples for fitting. The test set is used only once, for the final evaluation.

Each feature is standardised per detector using the mean $\mu_{s,f}$ and standard deviation $\sigma_{s,f}$ of the training period:

$$x'_{s,f} = \frac{x_{s,f} - \mu_{s,f}}{\sigma_{s,f}}$$

Per-detector scaling matches the per-detector labels: a standardised value of +1 means “one standard deviation busier than this detector’s norm”. With a single global scaling, the same input value would correspond to different classes at different detectors; in our experiments this reduced logistic regression from 87.2% to 69.4% accuracy.

C. Baselines

Five reference predictors are evaluated on the same test set: (i) *majority class*; (ii) *persistence*, which assigns the class of the most recent reading; (iii) *15-min average*, which assigns the class of the mean of the last three readings; (iv) *linear trend*, which extrapolates the last change one step ahead; and (v) *logistic regression* on the 72 flattened inputs, fitted on a random subset of 200,000 training windows. Rules (ii)–(iv) apply each detector’s own thresholds to the unscaled flow.

D. Artificial Neural Network

The ANN is a multi-layer perceptron on the flattened window ($24 \times 3 = 72$ inputs): fully connected layers of 128, 64 and 32 ReLU units, dropout of 0.2 after the first layer and a three-way softmax output, with 19,779 parameters in total.

E. One-Dimensional Convolutional Neural Network

The 1-D CNN treats the window as a three-channel time series. Two convolutional layers with 32 filters are followed by max pooling, a convolutional layer with 64 filters, max pooling and a convolutional layer with 128 filters (kernel size 3, ReLU, same padding). Global average pooling, a dense layer of 128 units, dropout of 0.3 and a softmax output complete the model, which has 51,235 parameters. Convolutions along time let the network detect local patterns such as rising or falling flow wherever they occur in the window.

F. Hybrid Quantum Neural Network

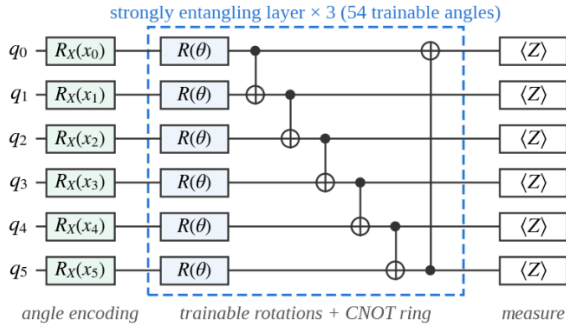


Fig. 2. Six-qubit variational circuit of the QNN. Inputs are encoded as RX rotation angles, followed by three strongly entangling layers (general rotations and a ring of CNOT gates) and Pauli-Z measurements.

The QNN places a simulated variational circuit [14], [15] in front of a small classical network (Fig. 2). Because a six-qubit circuit accepts six inputs, each 72-value window is first compressed by principal component analysis to six components, which retain 92.8% of the variance, and each component is scaled to an angle in $[0, \pi]$. The angles are loaded by RX rotations, transformed by three strongly entangling layers with 54 trainable rotation angles, and read out as the six expectation values $\langle Z \rangle \in [-1, 1]$. These feed dense layers of 64 and 32 ReLU units (dropout 0.3) and a softmax output, for 2,681 parameters in total. The circuit is simulated exactly with PennyLane’s *default.qubit* device [16], and gradients are obtained by backpropagation through the simulation inside TensorFlow [19]; the batch of samples is simulated in a single call by parameter broadcasting.

Circuit simulation is far more expensive than a dense layer, so the QNN is trained on a random subset of 50,000 fitting windows and 10,000 validation windows, drawn within each period so the chronological split is preserved; it is still evaluated on the full test set. To isolate the effect of the quantum layer, a **classical control** replaces the circuit with a dense layer of six tanh units (whose outputs, like $\langle Z \rangle$, lie in $[-1, 1]$) and is otherwise identical: same PCA inputs, training subset, classifier head and training procedure.

G. Training and Evaluation

All networks are trained with the Adam optimiser [20] and sparse categorical cross-entropy loss. The ANN and CNN use a batch size of 1,024 and a learning rate of 10^{-3} , the QNN and its control a batch size of 64 and a learning rate of 2×10^{-3} . Early stopping monitors validation loss with a patience of three epochs and restores the best weights (maximum 30, 25 and 20 epochs for the ANN, CNN and QNN). The random seed is fixed to 42. Models were implemented in TensorFlow 2.21 with Keras 3 [19] and PennyLane 0.45 [16], and trained on a CPU. Performance is measured by accuracy, macro-averaged precision, recall and F1 score, and per-class intersection over union (IoU, the Jaccard index). Accuracy is also computed separately for every detector. Differences between two models on the same test samples are tested with McNemar’s test [21].

V. RESULTS

A. Overall Performance

TABLE I
TEST-SET PERFORMANCE (606,390 WINDOWS)

Model	Accuracy (%)	Macro-F1	Mean IoU
1-D CNN	89.18	0.8911	0.8073
ANN	88.99	0.8893	0.8044
15-min average (rule)	88.08	0.8804	0.7904
Persistence (rule)	87.28	0.8721	-
Logistic regression	87.24	0.8717	-
QNN (6 qubits)	86.25	0.8625	0.7633
Classical control (6 units)	86.13	0.8610	0.7613
Linear trend (rule)	80.66	0.8043	-
Majority class	33.63	0.1678	-

Table I and Fig. 3 summarise the results. The baselines confirm how persistent traffic is at a five-minute horizon: always predicting the majority class gives 33.63%, whereas simply assigning the class of the latest reading gives 87.28%, and averaging the last three readings raises this to 88.08%. Extrapolating the last change (linear trend) performs worse, at 80.66%, because single five-minute changes are dominated by noise. Logistic regression (87.24%) does not improve on persistence, indicating that a linear model mostly recovers the persistence rule.

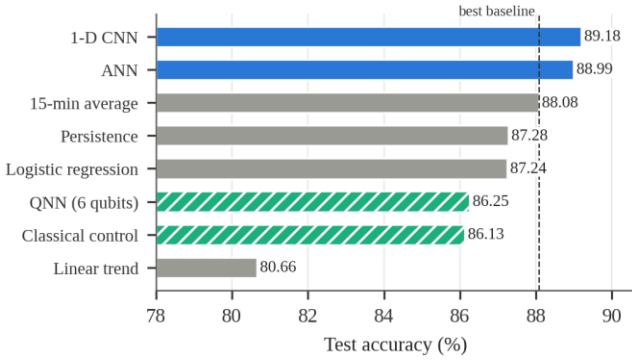


Fig. 3. Test accuracy of all models. Blue: ANN and CNN; hatched green: QNN and its classical control; grey: baselines. The dashed line marks the best baseline (15-min average).

The 1-D CNN achieves the best result, 89.18% accuracy and a macro-F1 of 0.8911, followed by the ANN with 88.99% and 0.8893. These exceed the best rule by 1.11 and 0.91 percentage points. The gains are small in absolute terms but highly consistent: on the paired test samples, McNemar’s test gives $p < 10^{-150}$ for both networks against the 15-min rule and $p \approx 10^{-16}$ for the CNN against the ANN. Early stopping selected epoch 16 of 19 for the ANN and epoch 15 of 18 for the CNN, and training and validation curves stayed close, with no sign of overfitting.

B. Error Structure

	ANN			1-D CNN			QNN		
	Low	Med	High	Low	Med	High	Low	Med	High
True Low	94.6	5.2	0.1	95.4	4.5	0.1	93.3	6.5	0.2
True Med	3.9	82.2	14.0	4.5	82.6	12.9	4.8	81.3	13.9
True High	0.0	10.0	90.0	0.0	10.6	89.4	0.1	15.9	84.0
	Low	Med	High	Low	Med	High	Low	Med	High

Fig. 4. Confusion matrices on the test set (row percentages).

Fig. 4 shows that errors occur almost exclusively between neighbouring classes: at most 0.2% of Low windows are predicted as High, and at most 0.1% of High windows as Low. Medium is the hardest class for every model (recall 81–83%, F1 0.83 for the ANN and CNN versus 0.95 for Low), because it is bounded by a threshold on both sides and traffic near either threshold can tip into the neighbouring class within five minutes. Table II lists per-class F1 scores and IoU.

TABLE II
PER-CLASS F1 SCORE AND IOU

Model	F1 Low / Med / High	IoU Low / Med / High
1-D CNN	0.955 / 0.834 / 0.885	0.914 / 0.715 / 0.793
ANN	0.954 / 0.831 / 0.883	0.912 / 0.710 / 0.791
15-min average	0.949 / 0.819 / 0.873	0.903 / 0.693 / 0.775
QNN	0.942 / 0.796 / 0.850	0.890 / 0.661 / 0.739
Classical control	0.943 / 0.794 / 0.846	0.892 / 0.658 / 0.734

C. Per-Detector Analysis

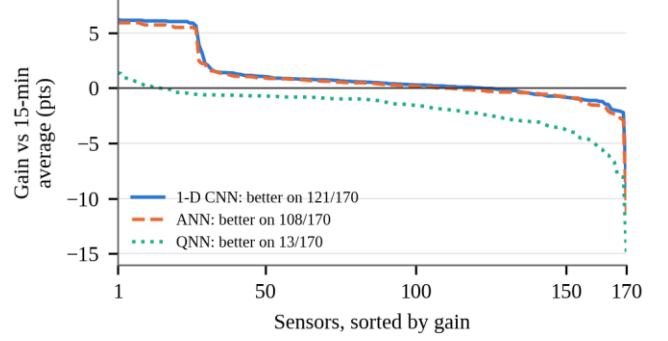


Fig. 5. Per-detector accuracy gain over the 15-min average rule, sorted from largest gain to largest loss for each model.

Fig. 5 compares each model with the 15-min rule detector by detector. The CNN is more accurate on 121 of the 170 detectors and the ANN on 108, with gains of up to about 6 percentage points on busy detectors with regular daily patterns. Median per-detector accuracy is 88.2% for both networks versus 87.7% for the rule. The largest loss occurs at detector 155 (−9.0 points for the CNN, −11.2 for the ANN), one of the two quietest detectors, whose mean flow rose by about 58% (from 16.8 to 26.5 vehicles per five minutes) between the training and test periods. The persistence-type rule adapts to such a shift automatically, whereas the networks rely on patterns learned under the earlier regime. Detectors 155 and 147, the two with the lowest traffic volume, are the hardest for every model.

D. Quantum Neural Network

The QNN reaches 86.25% accuracy (macro-F1 0.8625), below both the ANN and CNN and below the persistence rule. Its size-matched classical control reaches 86.13%. The difference of 0.12 percentage points is statistically detectable on 606,390 paired samples (McNemar’s $p \approx 1.6 \times 10^{-7}$) but negligible in practical terms, and it comes from a single training run of each model. The QNN’s main weakness is the High class: it labels 15.9% of High windows as Medium, compared with about 10% for the ANN and CNN (Fig. 4), and it beats the 15-min rule on only 13 detectors. Training the QNN took 79.8 min on a CPU, compared with seconds for the classical control.

VI. DISCUSSION

Strength of persistence. The central finding is that five-minute-ahead congestion classes are largely determined by current conditions: persistence-type rules reach about 88%, and the best network adds about one percentage point. Reporting learned models together with such baselines, and under a chronological split with training-only thresholds and scaling, is therefore essential. Randomly shuffling overlapping windows, or selecting the stopping epoch on the test set, would inflate accuracy without reflecting genuine predictive skill.

ANN vs CNN. The CNN’s small but significant advantage over the ANN is consistent with its inductive bias: convolutions along time detect local trends in flow, occupancy and speed regardless of their position in the window, while the ANN must

learn each position separately. Both networks improve mainly on windows near class thresholds, where the joint trend of the three features carries information that the flow value alone does not.

The quantum layer. Under a like-for-like comparison, the six-qubit circuit behaves as a trainable feature map that is neither better nor worse than a six-unit classical layer. The QNN's gap to the ANN and CNN is explained by its restricted input (six principal components, which lose the fine detail of the most recent readings that the persistence rule exploits) and its smaller training set, both imposed by the cost of classical simulation, rather than by the quantum layer itself. These findings are consistent with the current limits of near-term quantum machine learning [17], [18]; they do not rule out benefits from larger circuits, richer encodings or quantum hardware, but they provide no evidence of an advantage for this task.

Limitations. The study uses a single dataset, a single five-minute horizon and one training run per model; repeated runs with different seeds would quantify run-to-run variation, which matters most for the small QNN-versus-control difference. The models see only one detector's recent history, without time of day or information from neighbouring detectors; adding time of day to the ANN in preliminary tests made no measurable difference at this horizon. Finally, detectors with very low volume or a change in traffic between the training and test periods remain difficult for all learned models.

VII. CONCLUSION AND FUTURE WORK

This paper compared an ANN, a 1-D CNN and a hybrid QNN for five-minute-ahead congestion classification of 170 freeway detectors in PeMS08, under a leakage-free chronological protocol with per-detector labels and scaling. Simple persistence-type rules already achieve 88.08% accuracy. The 1-D CNN (89.18%) and ANN (88.99%) improve on the best rule by about one percentage point and on most detectors, with the CNN performing best. The six-qubit QNN (86.25%) performs on par with a size-matched classical control, showing no measurable quantum advantage in this setting.

Future work will extend the comparison to longer horizons (15–60 minutes), where persistence is weaker and learned models have more room to help; add time-of-day and neighbouring-detector information through graph-based models [9], [10], [11]; repeat all experiments over several random seeds; and evaluate QNNs with more qubits, data re-uploading encodings and execution on quantum hardware.

REFERENCES

- [1] California Department of Transportation, "Caltrans Performance Measurement System (PeMS)." [Online]. Available: <https://pems.dot.ca.gov/>
- [2] E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias, "Short-term traffic forecasting: Where we are and where we're going," *Transportation Research Part C: Emerging Technologies*, vol. 43, pp. 3–19, 2014.
- [3] B. M. Williams and L. A. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal ARIMA process: Theoretical basis and empirical results," *Journal of Transportation Engineering*, vol. 129, no. 6, pp. 664–672, 2003.
- [4] C.-H. Wu, J.-M. Ho, and D. T. Lee, "Travel-time prediction with support vector regression," *IEEE Transactions on Intelligent Transportation Systems*, vol. 5, no. 4, pp. 276–281, 2004.
- [5] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 2, pp. 865–873, 2015.
- [6] N. G. Polson and V. O. Sokolov, "Deep learning for short-term traffic flow prediction," *Transportation Research Part C: Emerging Technologies*, vol. 79, pp. 1–17, 2017.
- [7] X. Ma, Z. Dai, Z. He, J. Ma, Y. Wang, and Y. Wang, "Learning traffic as images: A deep convolutional neural network for large-scale transportation network speed prediction," *Sensors*, vol. 17, no. 4, p. 818, 2017.
- [8] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proc. AAAI Conference on Artificial Intelligence*, 2017, pp. 1655–1661.
- [9] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.
- [10] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, pp. 3634–3640.
- [11] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 33, no. 1, 2019, pp. 922–929.
- [12] M. Schuld, I. Sinayskiy, and F. Petruccione, "The quest for a quantum neural network," *Quantum Information Processing*, vol. 13, no. 11, pp. 2567–2586, 2014.
- [13] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, pp. 195–202, 2017.
- [14] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, "Quantum circuit learning," *Physical Review A*, vol. 98, no. 3, p. 032309, 2018.
- [15] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, "Circuit-centric quantum classifiers," *Physical Review A*, vol. 101, no. 3, p. 032308, 2020.
- [16] V. Bergholm et al., "PennyLane: Automatic differentiation of hybrid quantum-classical computations," *arXiv:1811.04968*, 2018.
- [17] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [18] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, p. 4812, 2018.
- [19] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.